

Hallazgos de análisis externo — ERSo 2026-001

- Hallazgos de análisis externo — ERSo 2026-001
 - Cartera de identidad y custodia de claves en hardware
 - 1. Resumen
 - 2. Proyectos citados en este informe
 - 3. Hallazgos
 - 4. Aplicación a los criterios de aceptación de la ERSo 001
 - 5. Diferencias de diseño frente a nuestro proyecto
 - 6. Qué aprender y qué hacer con esto
 - 7. Límites de este informe

Hallazgos de análisis externo — ERSo 2026-001

Cartera de identidad y custodia de claves en hardware

Proyecto propio	Identidad Digital Soberana SSI (/home/geovani/Descargas/generic/bitacora/new_project)
Fecha del análisis	2 de octubre de 2026
Carpetas analizadas	/home/geovani/Workspace/Wallet/Wallet, ../Verifier, ../Issuer, ../Trust, ../librerias
Versión del informe	Actualizado el 2 de octubre de 2026: se añaden los hallazgos de la carpeta librerias y el análisis a fondo del gestor de listas de confianza
Cobertura de esta ERSo	Alta (cartera, emisor, bibliotecas y especificación); contraste en Trust y en las bibliotecas de firma remota
Método	Lectura de código y documentación; no se compiló ni se ejecutó ninguno de los proyectos externos

1. Resumen

La ERSo 001 trata de que la clave nazca y viva en el hardware del dispositivo y de que el servidor **declare y verifique** el nivel real de protección. En los proyectos externos se ven las dos mitades: la **cartera Android** que crea y consulta las claves (Wallet/av-app-android-wallet-ui, Wallet/eudi-app-android-wallet-ui) y el **emisor** que sí verifica la key attestation del lado del servidor (Issuer/eudi-srv-pid-issuer, Issuer/eudi-srv-web-issuing-eudiw-py). Un proyecto de Trust (Trust/eudi-srv-web-trustprovider-signer-java) muestra el modelo contrario, con la clave custodiada en el servidor. No se encontró en la cartera ninguna verificación de la attestation del hardware. **Complemento (carpeta librerias)**: la biblioteca de la cartera confirma desde su propio código que StrongBox se solicita pero puede abandonarse sin aviso al usuario, y muestra cómo se crean las claves y qué datos de attestation se piden al emisor. Las bibliotecas de firma remota refuerzan el contraste con la custodia local.

2. Proyectos citados en este informe

Carpeta	Proyecto	Qué es
Wallet	av-app-android-wallet-ui	App Android de verificación de edad (versión derivada de la anterior); incluye el módulo vdr-extension-logic
Wallet	eudi-app-android-wallet-ui	App Android de la cartera EUDI (referencia)
Issuer	eudi-srv-pid-issuer	Emisor de credenciales (Kotlin/Spring)
Issuer	eudi-srv-web-issuing-eudiw-py	Emisor web con revocación (Python/Flask)
Verifier	eudi-wallet-rfcs	Especificaciones EWC (RFC 001 a 013)
Trust	eudi-srv-trust-validator	Validador de confianza de cadenas de certificados (Kotlin/Spring)
Trust	eudi-srv-web-trustprovider-signer-java	Firma remota de documentos, estándar CSC (Java)
librerias	eudi-lib-android-wallet-core	Biblioteca Android de la cartera EUDI: claves, documentos, confianza, estado, registro de transacciones
librerias	eudi-lib-android-wallet-document-manager	Gestor de documentos de la cartera (estados, políticas de credencial, claves)
librerias	eudi-lib-jvm-openid4vci-kt	Biblioteca de OpenID4VCI 1.0 (emisión, DPoP, attestation, diferida, notificación)
librerias	eudi-lib-jvm-rqes-csc-kt	Cliente CSC para firma remota cualificada
librerias	eudi-srv-web-walletdriven-rpcentric-signer-qtsp-java	Servidor QTSP de firma remota con autenticación OpenID4VP

3. Hallazgos

La columna *Criterio* indica a qué criterio de aceptación de la ERSo se aplica el hallazgo (el detalle está en la sección 4).

#	Proyecto (carpeta/proyecto)	Archivo o lugar	Qué se encontró
1	Wallet/av-app-android-wallet-ui	business-logic/src/main/java/eu/europa/ec/businesslogic/controller/crypto/KeystoreController.kt	KeystoreController usa Android Keystore (AES-GCM). getSecurityLevel(alias) lee KeyInfo.securityLevel (API 31+) y devuelve STRONGBOX, TEE, SOFTWARE o UNKNOWN; en versiones anteriores usa isInsideSecureHardware.
2	Wallet/av-app-android-wallet-ui	.../KeystoreController.kt (rotateKey , cleanupOldKey)	Rotación de la clave del Keystore con versión (getKeyVersion) y borrado de la clave anterior.
3	Wallet/av-app-android-wallet-ui Wallet/eudi-app-android-wallet-ui	core-logic/src/dev/.../config/WalletCoreConfigImpl.kt (configureDocumentKeyCreation)	useStrongBoxForKeys = true. En av-app-android-wallet-ui: userAuthenticationRequired = true, timeout 10 s. En eudi-app-android-wallet-ui: userAuthenticationRequired = false, timeout 30 s.
4	Wallet/eudi-app-android-wallet-ui	authentication-logic/.../controller/throttle/PinThrottleController.kt	Bloqueo por intentos fallidos de PIN (recordFailure , getState). Existe solo en eudi-app-android-wallet-ui, no en av-app-android-wallet-ui.
5	Wallet/av-app-android-wallet-ui Wallet/eudi-app-android-wallet-ui	network-logic/.../repository/WalletAttestationRepository.kt	Pide al proveedor de la cartera POST /wallet-instance-attestation/jwk y POST /wallet-unit-attestation/jwk-set (con nonce).
6	Wallet/av-app-android-wallet-ui Wallet/eudi-app-android-wallet-ui	core-logic/.../provider/WalletCoreAttestationProvider.kt	Entrega la attestation de la instancia y de las claves a la biblioteca de la cartera; no la verifica.
7	Issuer/eudi-srv-pid-issuer	src/main/kotlin/.../adapter/out/proof/ValidateJwtProofWithKeyAttestation.kt (línea ~114)	El emisor exige key_attestation en la cabecera de la prueba JWT ("JWT Proof must contain key_attestation") y comprueba que su nonce coincide con el de la prueba (línea ~142).
8	Issuer/eudi-srv-pid-issuer	.../adapter/out/proof/VerifyKeyAttestation.kt	Exige cadena x5c, firma, claims iat / attested_keys / exp, proveedor de cartera de confianza (ensureTrustWalletProvider) y compara keyStorage y userAuthentication con lo exigido (líneas ~140 a 152). Consulta además el estado de la clave en una lista de estado (~159).
9	Issuer/eudi-srv-pid-issuer	application.properties (issuer.cnonce.expiration=PT5M) y .../adapter/out/nonce/GenerateNonceAndEncryptWithNimbus.kt	El nonce se entrega cifrado (JWE) y vence en 5 minutos.
10	Issuer/eudi-srv-web-issuing-eudiwpy	app/route_oidc.py (decode_verify_attestation , verify_wua_jwt_with_x5c , líneas ~396 a 598)	Versión Python de la verificación de attestation de las pruebas attestation y jwt.
11	Verifier/eudi-wallet-rfcs	ewc-rfc004-individual-wallet-attestation.md (§3.1, §3.2)	Define la <i>Wallet Unit Attestation</i> (WUA) y su JWT de prueba de posesión.
12	Trust/eudi-srv-trust-validator	src/main/kotlin/.../port/input/trust/IsChainTrustedUseCase.kt (VerificationContextTO)	El contexto WalletProviderAttestation permite preguntar si la cadena de un proveedor de cartera es confiable.
13	Trust/eudi-srv-web-trustprovider-signer-java	server/app/.../rssp y server/sa/... (/csc/v1/signatures/signHash , SADProperties.java)	Firma remota (CSC): la clave la custodia el servicio (HSM opcional) y se autoriza con <i>Signature Activation Data</i> .
14	librerías/eudi-lib-android-wallet-core	wallet-core/src/main/java/eu/europa/ec/eudi/wallet/EudiWallet.kt (ensureStrongBoxIsSupported , líneas ~760 a 770)	Si el dispositivo no soporta StrongBox y useStrongBoxForKeys es true, la biblioteca solo escribe un mensaje informativo y pone la opción en false ("Setting EudiWalletConfig.useStrongBoxForKeys to false").

#	Proyecto (carpeta/proyecto)	Archivo o lugar	Qué se encontró
15	librerias/eudi-lib-android-wallet-core	wallet-core/.../EudiWalletConfig.kt (líneas ~144 a 146 y ~775 a 808)	userAuthenticationRequired, userAuthenticationTimeout y useStrongBoxForKeys (por defecto true) son los tres ajustes de creación de claves de documento.
16	librerias/eudi-lib-android-wallet-core	wallet-core/.../provider/DefaultWalletKeyManager.kt	Crea un AndroidKeystoreSecureArea sobre un archivo en noBackupFilesDir (wallet-attest.bin) y genera la clave con un desafío aleatorio de 32 bytes creado localmente (SecureRandom).
17	librerias/eudi-lib-android-wallet-core	CustomizeSecureArea.md	Permite sustituir la gestión de claves con interfaces propias (SecureArea, KeyUnlockData) de la biblioteca Multipaz.
18	librerias/eudi-lib-android-wallet-document-manager	document-manager/.../DocumentManager.kt y CreateDocumentSettings.kt	createDocument usa un SecureAreaRepository; el documento nace como UnsignedDocument y se crean numberOfCredentials credenciales con una credentialPolicy (RotateUse por defecto o OneTimeUse).
19	librerias/eudi-lib-jvm-openid4vci-kt	src/main/kotlin/.../openid4vci/KeyAttestationJWT.kt	Claims de la key attestation: iat, exp, attested_keys, key_storage y user_authentication (listas de resistencia a ataques).
20	librerias/eudi-lib-jvm-openid4vci-kt	.../openid4vci/ClientAttestation.kt e internal/AttestationBasedClientAuthentication.kt	La cartera se autentica ante el emisor con una attestation del cliente.
21	librerias/eudi-lib-android-wallet-core	wallet-core/.../transactionLogging/ (TransactionLogger, TransactionLogManager, TransactionLogExport)	Registro de transacciones de la cartera y exportación.
22	librerias/eudi-lib-jvm-rqes-csc-kt librerias/eudi-srv-web-walletdriven-rpcentric-signer-qtsp-java	eudi-lib-jvm-rqes-csc-kt (README) y eudi-srv-web-walletdriven-rpcentric-signer-qtsp-java	La biblioteca CSC implementa info, credentials/list, credentials/info y signatures/signHash con autorización OAuth 2.0 de servicio y de credencial; el QTSP sigue CSC API v2.0 con autenticación OpenID4VP.

4. Aplicación a los criterios de aceptación de la ERSO 001

Para cada criterio: lo que exige el PDF de la ERSO, dónde está nuestra evidencia y qué hallazgos externos se le aplican.

Criterio 1. La instancia de cartera queda registrada y activa, asociada al ciudadano.

Evidencia que pide la ERSO	Registro de activación.
Nuestra evidencia	INFORME-FINAL-ERSO-2026-001 §16
Hallazgos aplicables	#5 (Wallet/av-app-android-wallet-ui, Wallet/eudi-app-android-wallet-ui); #9 (Issuer/eudi-srv-pid-issuer); #11 (Verifier/eudi-wallet-rfcs); #20 (librerias/eudi-lib-jvm-openid4vci-kt); #21 (librerias/eudi-lib-android-wallet-core)
Qué aporta el análisis	La activación con attestation y nonce tiene una contraparte real: la cartera pide una attestation al proveedor y el emisor la exige; el registro de activación sigue siendo propio de nuestro Wallet Backend.

Criterio 2. El par de claves se genera dentro del hardware seguro y la clave privada no puede extraerse ni exportarse.

Evidencia que pide la ERSO	Prueba de no exportabilidad.
Nuestra evidencia	INFORME-FINAL-ERSO-2026-001 §17
Hallazgos aplicables	#1 (Wallet/av-app-android-wallet-ui); #2 (Wallet/av-app-android-wallet-ui); #3 (Wallet/av-app-android-wallet-ui, Wallet/eudi-app-android-wallet-ui); #13 (Trust/eudi-srv-web-trustprovider-signer-java); #15 (librerias/eudi-lib-android-wallet-core); #16 (librerias/eudi-lib-android-wallet-core); #17 (librerias/eudi-lib-android-wallet-core); #18 (librerias/eudi-lib-android-wallet-document-manager); #22 (librerias/eudi-lib-jvm-rqes-csc-kt, librerias/eudi-srv-web-walletdriven-rpcentric-signer-qtsp-java)
Qué aporta el análisis	La app Android muestra cómo se crean y consultan claves en el Keystore (nivel de seguridad, rotación). Es la plantilla para la prueba en un teléfono real que aún está pendiente. El proyecto de firma remota sirve de contraste: allí la clave la custodia el servidor. Las bibliotecas muestran que cada credencial tiene su propia clave en el área segura, con política de uso.

Criterio 3. El nivel de protección declarado coincide con el realmente disponible en el dispositivo.

Evidencia que pide la ERSO	Ficha técnica y verificación del dispositivo.
Nuestra evidencia	INFORME-FINAL-ERSO-2026-001 §18
Hallazgos aplicables	#1 (Wallet/av-app-android-wallet-ui); #3 (Wallet/av-app-android-wallet-ui, Wallet/eudi-app-android-wallet-ui); #5 (Wallet/av-app-android-wallet-ui, Wallet/eudi-app-android-wallet-ui); #6 (Wallet/av-app-android-wallet-ui, Wallet/eudi-app-android-wallet-ui); #7 (Issuer/eudi-srv-pid-issuer); #8 (Issuer/eudi-srv-pid-issuer); #9 (Issuer/eudi-srv-pid-issuer); #10 (Issuer/eudi-srv-web-issuing-eudiw-py); #11 (Verifier/eudi-wallet-rfcs); #12 (Trust/eudi-srv-trust-validator); #14 (librerias/eudi-lib-android-wallet-core); #15 (librerias/eudi-lib-android-wallet-core); #16 (librerias/eudi-lib-android-wallet-core); #19 (librerias/eudi-lib-jvm-openid4vci-kt); #20 (librerias/eudi-lib-jvm-openid4vci-kt)
Qué aporta el análisis	Es el criterio con más material: el emisor verifica la attestation (cadena x5c, nonce, nivel de almacenamiento y autenticación exigidos) y confirma que la app solo pide StrongBox sin verificarlo. La biblioteca de la cartera muestra que la degradación de StrongBox a TEE/software es automática, lo que justifica verificar el nivel en el servidor.

Criterio 4. La recuperación de cuenta se completa desde el backend sin acceder a la clave privada.

Evidencia que pide la ERSO	Prueba de recuperación.
Nuestra evidencia	INFORME-FINAL-ERSO-2026-001 §19
Hallazgos aplicables	#4 (Wallet/eudi-app-android-wallet-ui); #13 (Trust/eudi-srv-web-trustprovider-signer-java)
Qué aporta el análisis	Solo se encontró el bloqueo por intentos fallidos de PIN (en la app de referencia EUDI). No se halló recuperación de cuenta en los proyectos externos: este criterio se sustenta solo con nuestras pruebas. No se encontró recuperación de cuenta en las bibliotecas analizadas.

5. Diferencias de diseño frente a nuestro proyecto

Tema	Proyectos externos	Nuestro proyecto
Dónde se verifica el nivel de protección	El emisor verifica una attestation emitida por un proveedor de cartera (JWT con x5c)	Se verifica la cadena X.509 de Android Key Attestation (KeyDescription) contra una autoridad de laboratorio
Qué se compara	keyStorage y userAuthentication exigidos por el emisor	Nivel declarado frente a nivel verificado, con igualdad estricta (también se rechaza declarar de menos)
Autenticación del usuario por operación	Presente (userAuthenticationRequired)	No modelada
Recuperación de cuenta	No se encontró en la cartera	Código de recuperación, token de un solo uso y revocación de la cartera anterior

6. Qué aprender y qué hacer con esto

- Para la defensa de la ERSO 001: el equivalente real del KeyAttestationVerifier está en Issuer/eudi-srv-pid-issuer (VerifyKeyAttestation.kt); sirve para explicar qué cambia entre la attestation de Android y la WUA.
- Estudiar KeystoreController.getSecurityLevel para preparar la prueba en un teléfono real (KeyInfo.securityLevel).
- Considerar si se añade el **estado de la clave** (lista de estado) como idea futura, tal como hace Issuer/eudi-srv-pid-issuer.
- Decisión pendiente del equipo: autenticación por operación (biometría/PIN) y su efecto en el nivel exigido.
- Para la defensa: citar EudiWallet.ensureStrongBoxIsSupported como prueba de que no se puede confiar en lo que la aplicación declara del hardware.
- Idea a evaluar: que el desafío de la attestation venga del servidor (como en nuestro diseño) y no se genere localmente, como hace DefaultWalletKeyManager.

7. Límites de este informe

- Todo proviene de **lectura de código**; el comportamiento real no se verificó ejecutando.
- Se citan archivos y, cuando se anotó, números de línea (con ~ si son aproximados); las versiones de los proyectos son las que había en disco el 2 de octubre de 2026.
- "No encontré" significa que las búsquedas realizadas no lo hallaron, no que no exista en ninguna parte del código.
- Los proyectos de referencia EUDI se declaran a sí mismos como versiones de desarrollo no aptas para producción.